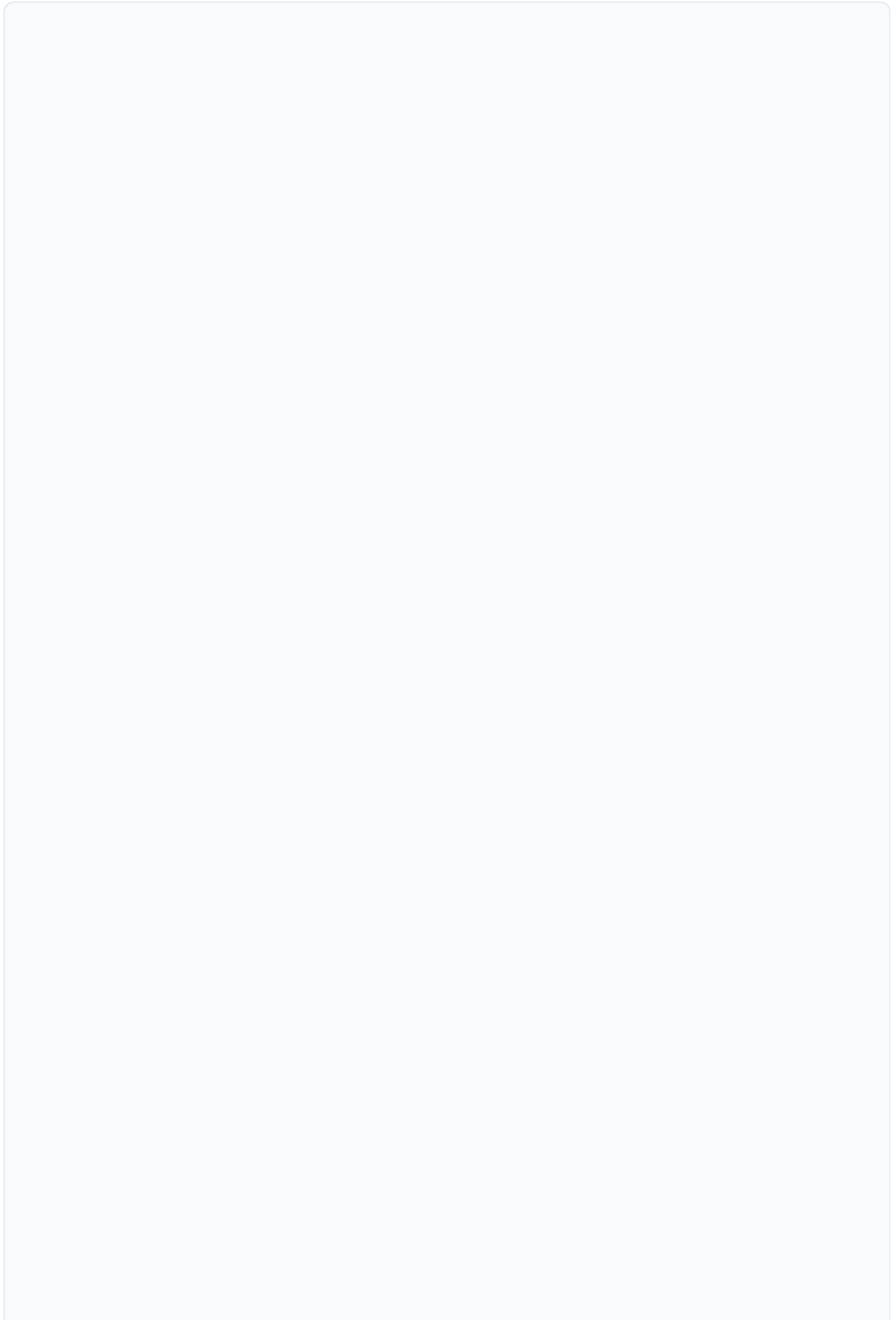


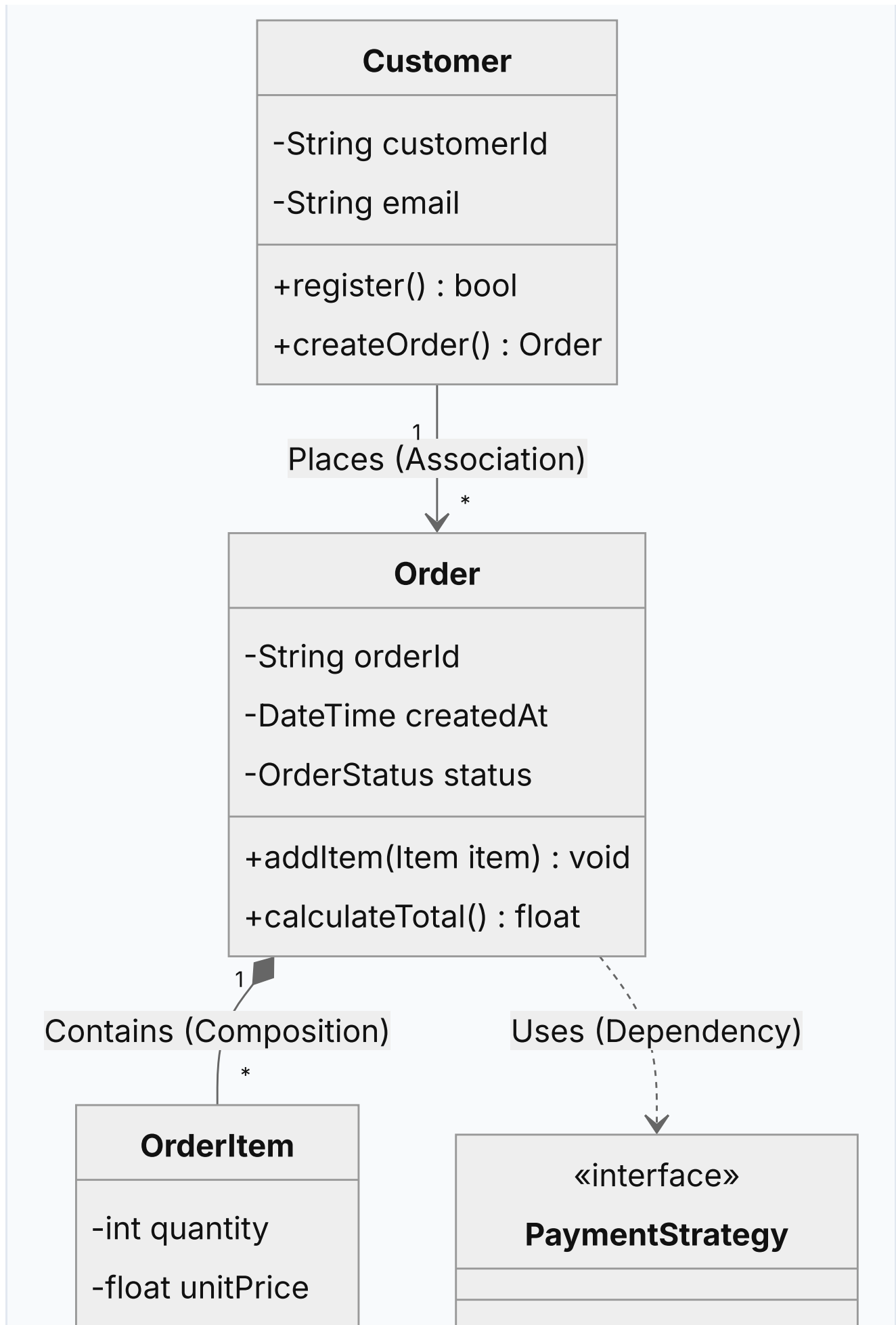
LLD Chapter 2: UML Modeling: Class, Sequence & State Diagrams

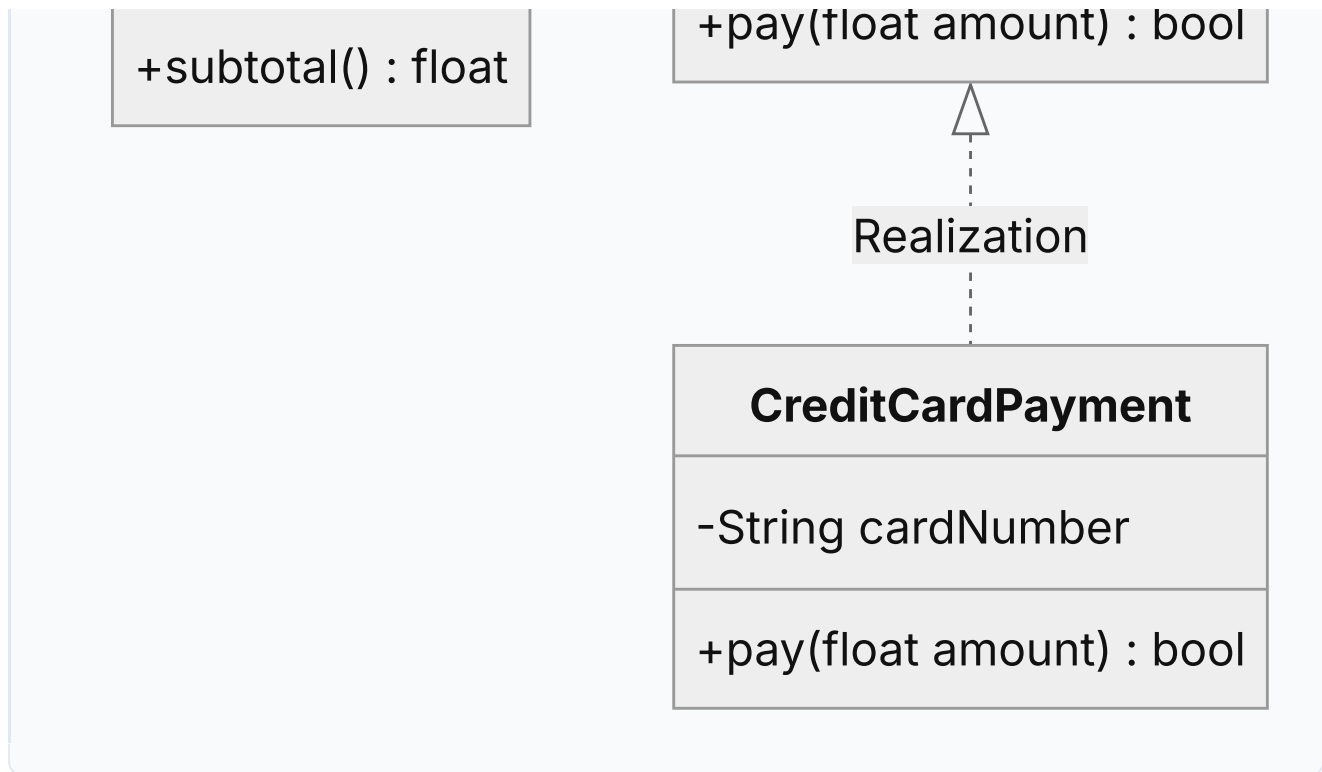
Core Learning Objective: Master the visual language of Software Engineering. Learn how to draft high-scoring Class Diagrams, Sequence Diagrams, and State Machines in under 5 minutes during a PBC Low-Level Design interview using Mermaid notation.

1. Class Diagrams: Structural Relationships

A Class Diagram models the static architecture of your system, showing classes, attributes, methods, and relationships.





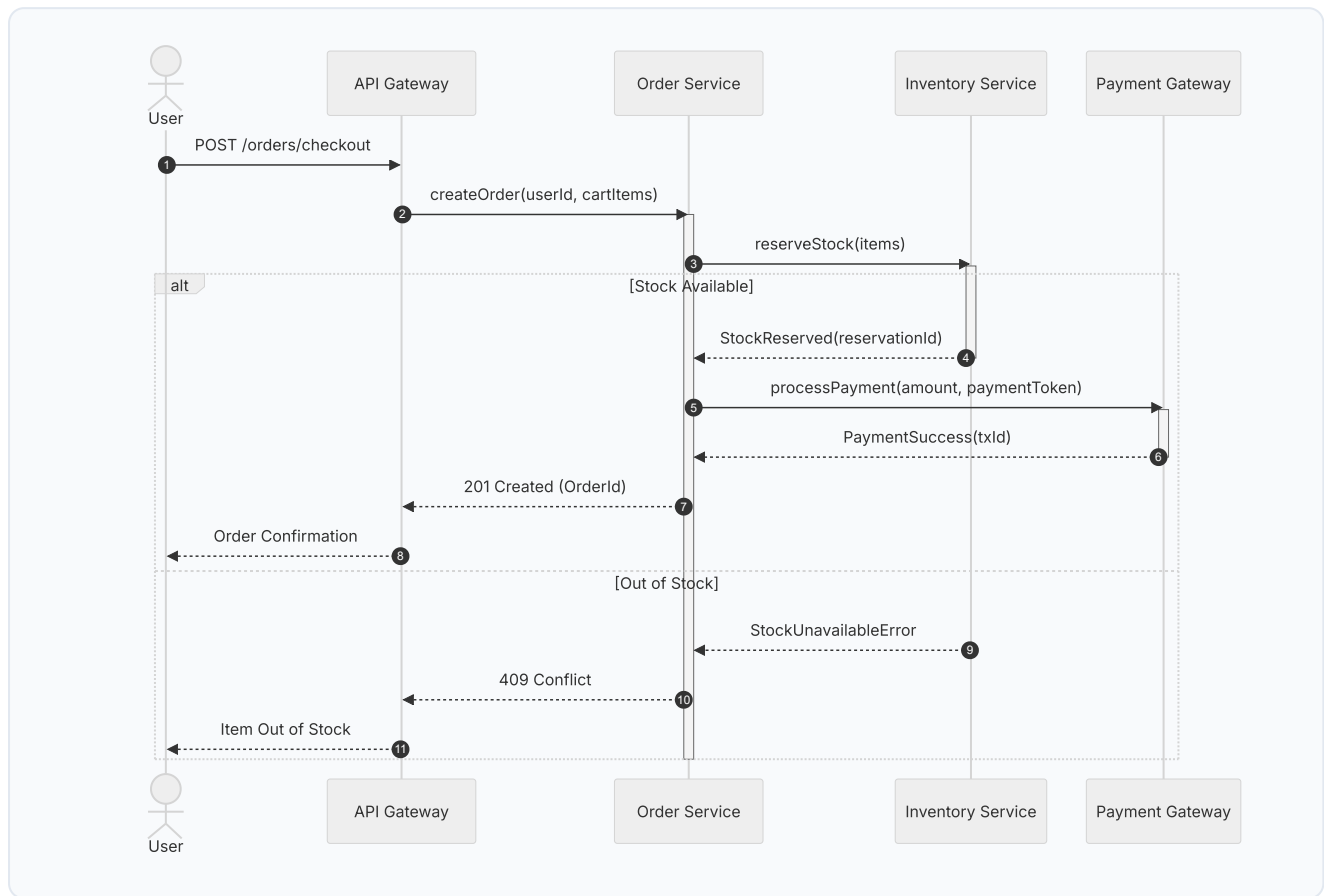


UML Relationship Taxonomy

Relationship	Symbol	Semantic Meaning	Lifecycle Coupling
Inheritance (Generalization)		Class <i>B</i> inherits from Class <i>A</i> ("is-a")	Permanent
Realization (Interface)		Class implements an abstract Interface/Protocol	Permanent
Composition		Whole-part relationship ("owns-a"). If parent dies, child dies!	Strict ownership (Child cannot exist without Parent)
Aggregation		Whole-part relationship ("has-a"). Child can exist independently.	Loose ownership (e.g. Department has Professors)
Association		Class <i>A</i> holds a reference to Class <i>B</i>	Reference holding
Dependency		Method in Class <i>A</i> receives Class <i>B</i> as transient parameter	Temporary runtime coupling

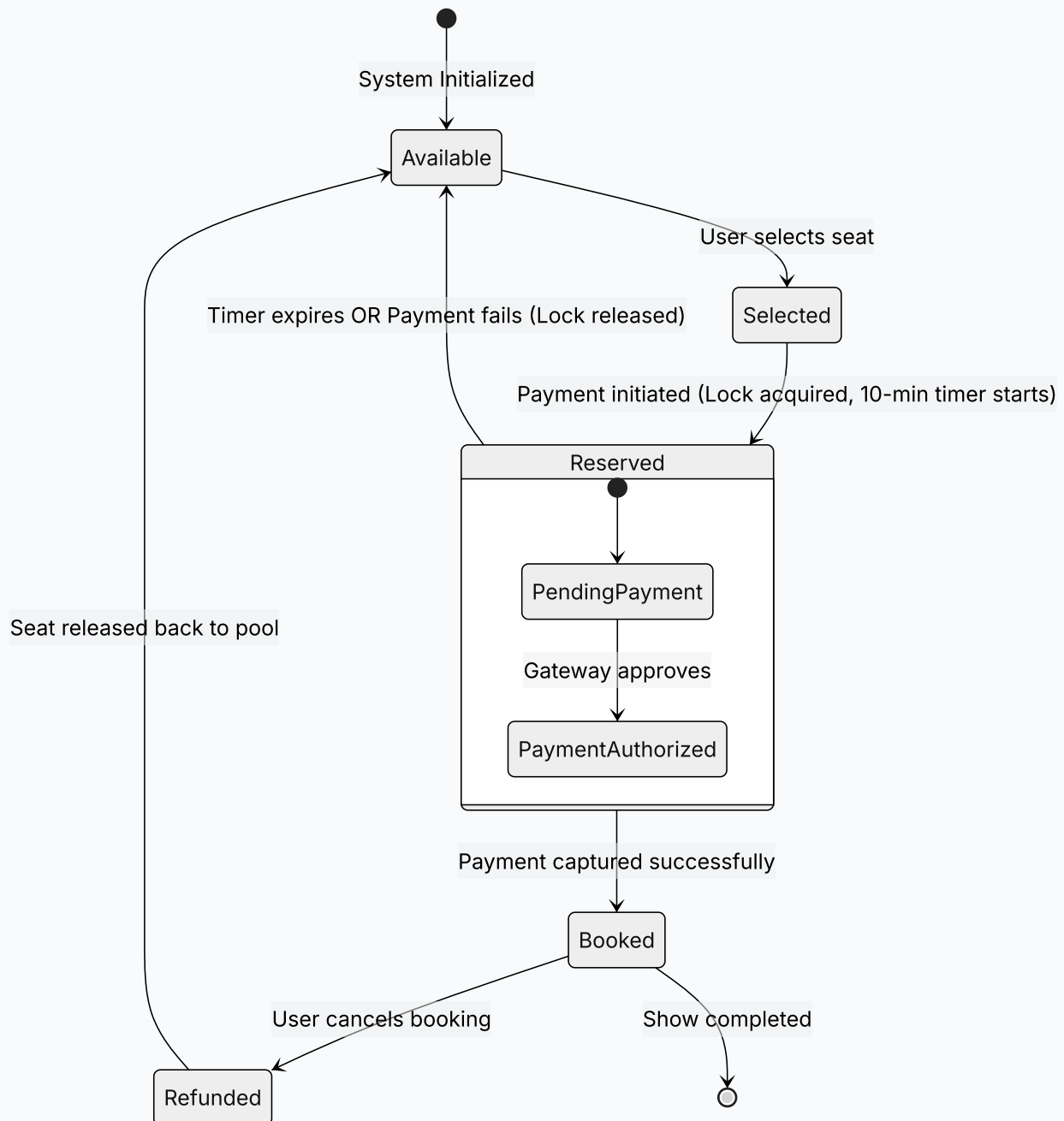
2. Sequence Diagrams: Behavioral Flow & Coordination

Sequence diagrams capture **time-ordered object interactions** across lifelines. Essential for explaining API request flows, payment checkout, and distributed locking.



3. State Machine Diagrams: Entity Lifecycle Transitions

State machines are required whenever an entity changes behavior depending on its current lifecycle phase (e.g., Vending Machine, Order status, Elevator state, Movie seat reservation).



4. The 5-Minute LLD Interview Drawing Strategy

During an interview, do not spend 20 minutes drawing exhaustive getter/setter details. Follow this **3-Pass Architecture Template**:

1. **Pass 1 (Core Entities)**: Draw 3 to 5 primary noun classes with their core responsibility.
2. **Pass 2 (Relationships & Multiplicity)**: Connect classes with correct lines (`*--` composition for components, `→` association for collaborators, `1 to *` multiplicity).

3. **Pass 3 (Behavior & Patterns):** Add interfaces for Strategy, Observer, or Factory patterns where flexibility is needed.

5. From Diagram to Code: What Each Arrow Means in Python

Interviewers check that your arrows mean what they say. The same six relationships, with the Python each one implies:

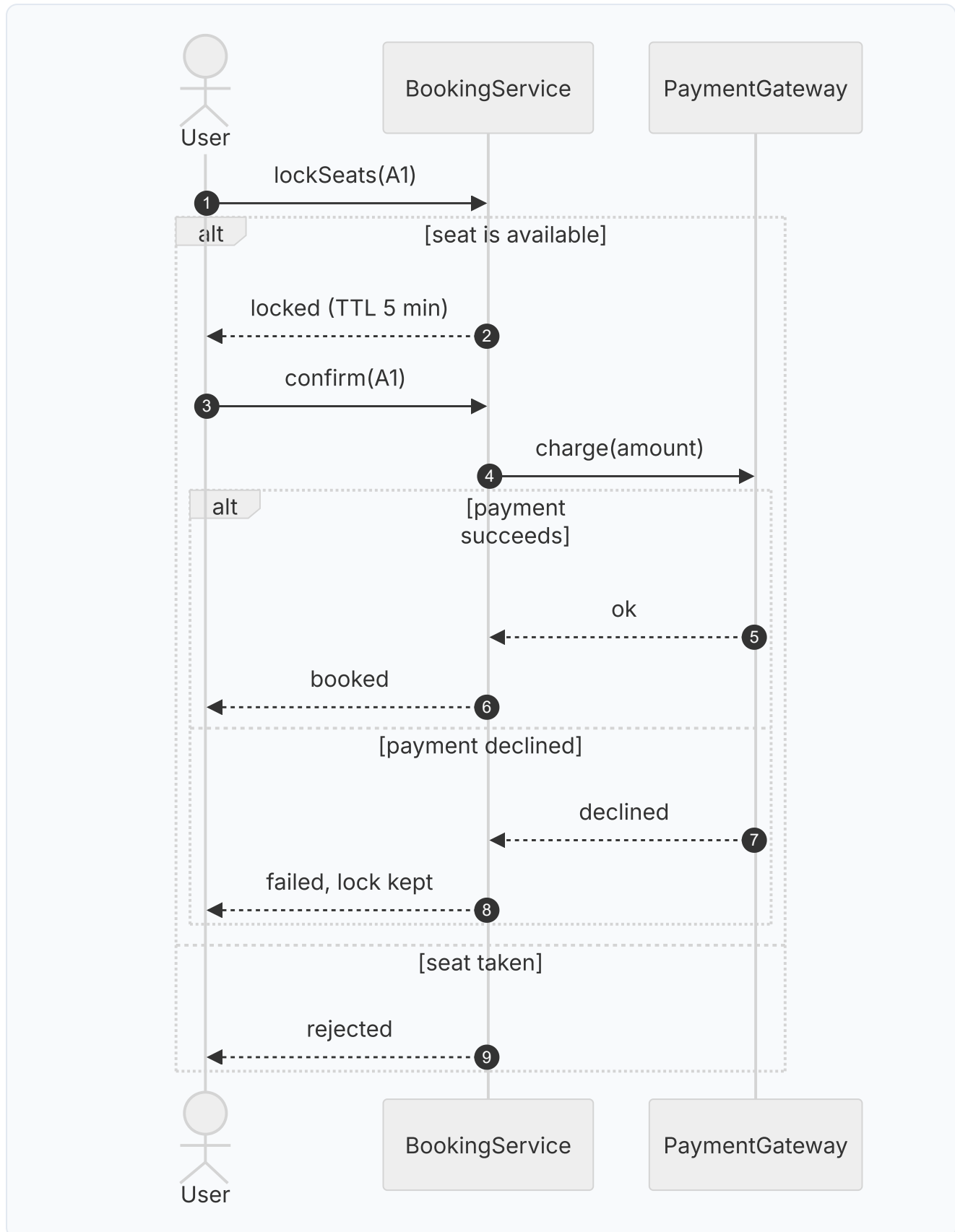
Relationship	Arrow (Mermaid)	Meaning	Lifetime rule	Python shape
Inheritance	<code>A < -- B</code>	B is an A	n/a	<code>class B(A)</code>
Realisation	<code>A <.. B</code>	B implements interface A	n/a	<code>class B(A)</code> where A is an ABC or Protocol
Association	<code>A --> B</code>	A holds a reference to B	independent	<code>self.b = b</code> passed in
Aggregation	<code>A o-- B</code>	A has B, B can exist elsewhere	B outlives A	<code>self.items = items</code> (shared list)
Composition	<code>A *-- B</code>	A owns B	B dies with A	<code>self.engine = Engine()</code> created inside
Dependency	<code>A ..> B</code>	A uses B briefly	none	B appears only as a parameter or local

The test that separates composition from aggregation is **who creates the part and who can still see it after the whole is gone.**

Multiplicity and navigability

Write multiplicities on the ends of an association (`1` , `0..1` , `*` , `1..*`). A `1..*` end means the constructor or a validation step must reject an empty collection; a `0..1` end means the attribute may be `None` and every use needs a check. Arrow direction shows who knows whom: `Order --> Customer` means an `Order` holds a customer reference but a `Customer` does not hold orders, so listing a customer's orders needs a query or a separate index.

Sequence diagram fragments you are expected to know



Use `alt/else` for exclusive branches, `opt` for an optional step, `loop` for repetition and `par` for concurrent work. A sequence diagram should show **who calls whom and in which order**, so draw

one for the single most important use case rather than for everything.

Five mistakes that cost marks

1. Drawing every getter and setter; show only behaviour that matters to the design.
 2. Using inheritance arrows for "has-a" relationships.
 3. Giving two classes a bidirectional association without saying which side owns it.
 4. Putting the same responsibility in several boxes; each class gets one sentence of purpose.
 5. A state diagram with an unreachable state or a state with no exit and no "terminal" mark.
-

Further Reading

- [UML diagrams reference](#)
 - [Mermaid sequence diagrams](#)
 - [Mermaid state diagrams](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. Composition versus aggregation in a class diagram?
- ▶ 2. What does a dashed arrow with a hollow head mean?
- ▶ 3. When is a sequence diagram more useful than a class diagram?
- ▶ 4. What does a state diagram model?